MS SQL Joins Part 3

By Don Schlichting

Introduction

The purpose of a Join is to combine query results from two or more related tables into one logical result set. If a database is designed for OLTP (Online Transaction Processing) or data entry, then many related tables will be used in the design rather than one large all encompassing table. By using many related tables, performance gains will be achieved for data input. The down side to this design is the difficulty in retrieving commingled record sets back from the multiple tables. Joins enable this data retrieval. In Part 1 of this series, additional benefits and reason for using Joins were discussed. In addition, examples and definitions of INNER Joins were explored. Part 2 in this series expanded beyond INNER Joins to include LEFT OUTER, RIGHT OUTER, and FULL OUTER Joins. This third and final article will cover Aliasing, Joining more than two tables, and Joins used in UPDATE statements.

Aliases

An Alias is a made up name assigned to an object to make it easier to work with. For example, if there was a table named "DivisonSalesResultsCube", an Alias called "DivSales" could be assigned. Then, anytime this tabled needed to be included in a TSQL statement, the short name DivSales could be used rather than its true long name. Aliases are not required for Joins, but Table Aliases are often used to make TSQL code easier to follow and more readable. In addition, many Join examples, including those in BOL, make use of them. The statement below shows an example of a Table Alias.

```
SELECT s.CustID, s.CustName, s.Address
FROM Sales s
```

The last "s" in the second line creates the Table Alias "s" for the table Sales. An Alias can be a letter or word. Then the new alias was used before each column name, specifying that the column should be returned from the "s", or Sales table. In this example, the entire Alias step could have been skipped and the query would have executed correctly. Aliases are never required, but often when working with multiple tables, specifying which table a column belongs to is required. In these cases, although an Alias is not required, they are often used. Below is an example of when specifying table names would be required.


```
SELECT c.CustID
FROM Customers c INNER JOIN Sales ON c.CustID = Sales.CustID
```

In this example, when returning the CustID, because it appears in both tables, a table name must be used. If the statement just said SELECT CustID, the SQL engine wouldn't know if it should return the value from the Customers table or the Sales table and an error would be returned. This example only had an Alias for the Customers table, but a more typical syntax would have included Aliases for both tables as shown below.

```
SELECT c.CustID
FROM Customers c INNER JOIN Sales s ON c.CustID = s.CustID
```

Joining Three or More Tables

In all the examples so far, only two tables were joined. However, the Join statement also supports three or more tables as show below.

```
SELECT Sales.*, Customers.*, Parts.*, Tax.*
FROM Sales
INNER JOIN Customers ON Sales.CustID = Customers.CustID
 INNER JOIN Parts ON Parts.PartID = Sales.PartID
 INNER JOIN Tax ON Tax.TaxID = Customers.TaxID
```

For each new table desired in the result set, add an additional JOIN statement. The position each JOIN appears in, (this example has Customers first), doesn't matter. In addition, INNER JOINS can be mixed and matched with OUTER JOINS in the same statement. The same statement using Aliases appears below.

```
SELECT s.*, c.*, p.*, t.*
FROM Sales s
INNER JOIN Customers c ON s.CustID = c.CustID
 INNER JOIN Parts p ON p.PartID = s.PartID
 INNER JOIN Tax t ON t.TaxID = c.TaxID
```

Updates

The next few examples use to tables, Customers and Sales. Execute the script below to create them.

```
CREATE TABLE [dbo].[Sales](
 [SalesDate] [datetime] NULL,
 [CustID] [int] NULL,
 [ProductID] [int] NULL,
 [Quantity] [int] NULL,
 [CustName] [varchar](50) COLLATE SQL_Latin1_General_CP1_CI_AS NULL,
 [SalesAmt] [money] NULL
) ON [PRIMARY]
GO
CREATE TABLE [dbo].[Customers](
 [CustID] [int] NULL,
 [CustomerName] [nchar](10) COLLATE SQL_Latin1_General_CP1_CI_AS NULL,
 [Addr] [nchar](10) COLLATE SQL_Latin1_General_CP1_CI_AS NULL,
 [City] [nchar](10) COLLATE SQL_Latin1_General_CP1_CI_AS NULL,
 [TotalSales] [varchar](50) COLLATE SQL_Latin1_General_CP1_CI_AS NULL
```

```
) ON [PRIMARY]
GO
```

Joins can also be used in update statements. The example below joins Sales to Customer on the Customer ID. The UPDATE will populate the Customer Name into the Sales table.

```
UPDATE Sales
SET Sales.CustName = Customers.CustomerName
FROM Sales INNER JOIN Customers ON Sales.CustID = Customers.CustID
```

Sometimes INNER JOIN examples will be written in the older style where tables are seperated by a comma and the ON is replaced by a WHERE clause as show below.

```
UPDATE Sales
SET Sales.CustName = Customers.CustomerName
FROM Sales, Customers
WHERE Sales.CustID = Customers.CustID
```

Both statements are functionally equivalent. This style can only be used on INNER JOINS though; OUTERS are not supported.

AGGRAGETS

Often the SUM or some other aggregate from a joined table will be needed in an UPDATE statement. For example, we could SUM a sales table, and then enter the grand total for each customer in the CUSTOMERS table under the column TotalSales. The statement below will use a subquery and a JOIN. Again, because the statement uses an INNER JOIN, it can be written in the old or new style of JOINS.

```
UPDATE Customers
SET TotalSales =
 (
SELECT SUM(SalesAmt)
FROM Sales
WHERE Sales.CustID = Customer.CustID
)
FROM Sales, Customers
```

```
UPDATE Customers
SET TotalSales =
 (
SELECT SUM(SalesAmt)
FROM Sales
WHERE Sales.CustID = Customers.CustID
)
FROM Sales INNER JOIN Customers ON Sales.CustID = Customers.CustID
```

Not to JOIN

There are times not to JOIN. Joins require processing overhead. Depending on the number of rows, Joins, and indexes, a multi join statement for reporting can have a negative impact

on a production server. If this occurs, try using multiple simple select statements that populate temp tables over several steps. It may be an improvement.

Conclusion

Joins allow tables that were designed for input productivity to still support reporting by merging data from multiple tables into one record set. The join syntax is fairly straight forward if good coding layout and naming conventions are used.